# Workload-driven evaluation

# Bibliography


- Culler et. al. – Chapter 4

# Until now…

- Parallel execution in modern microprocessors
- Parallel programming models
- Parallel algorithms design
- Improve the performance of parallel programs

# What's next?

- Measure the performance of parallel programs on a real parallel machine

# Single processor case

- Chose relevant workloads -> benchmark suites (standard)
- Measure
  - Absolute performance  (execution time, etc.)
  - Efficiency
- Evaluate new architectural feature through simulation -> expensive

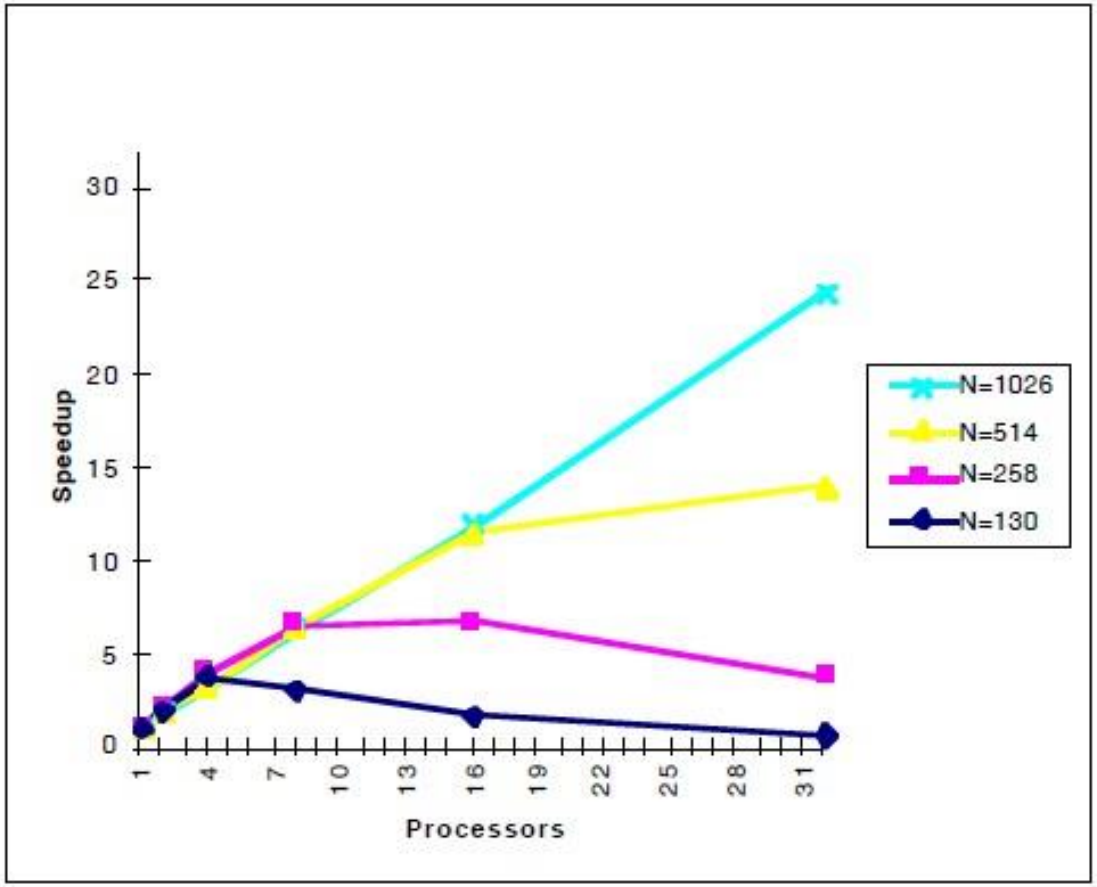
# What to measure?

- Absolute performance:
  - execution time
  - operations per second
- Overhead:
  - Total parallel overhead: $T_o = PT_p - T_s$
- Speedup
  - Exec time (1) / Exec time (p)
  - Ops per sec (p) /Ops per sec (1)
- Efficiency
  - performance/unit of resource
  - E=Speedup/P
- Cost: $PT_p$

# Speedup

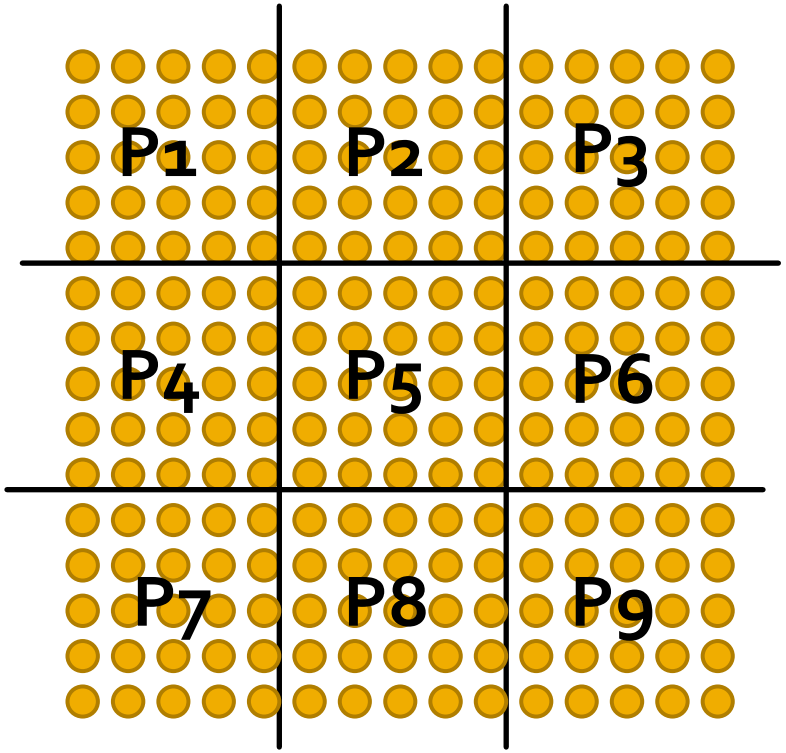
- Compare execution on p processors against:
  - Parallel version of a program running on one processor (makes yourself look good)
  - Best sequential program

# Parallel machine case



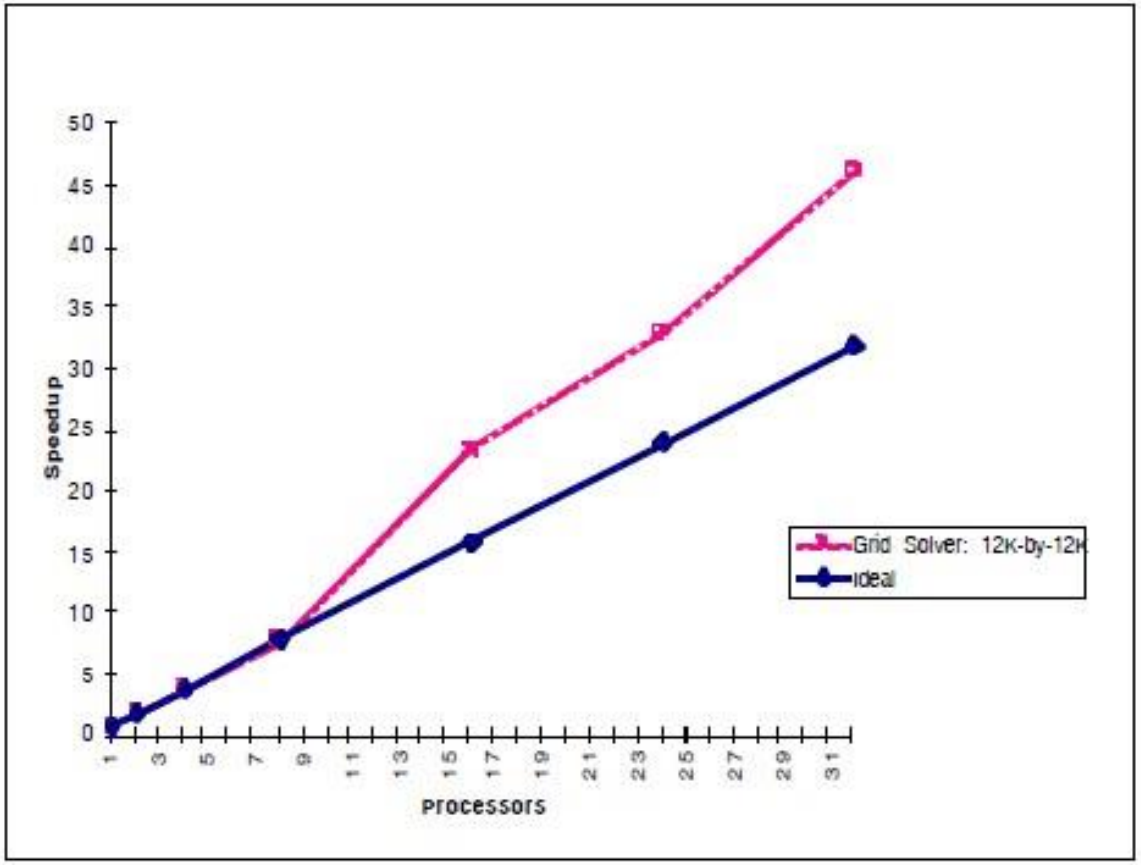

- Ocean execution on 32 processor SGI Origin 2000
- No benefit for small size problem

# Solver example




- Communication to computation ratio=$\frac{\sqrt{P}}{N}$
- If N is small -> $\frac{\sqrt{P}}{N}$ is high

# Superlinear speedup




- Problem is too large to fit on a smaller machine (e.g. data does not fit in cache)
- The amount of work done by the sequential algorithm is larger (e.g. exploratory decomposition)

# Understanding scaling

- Complex relationships among application parameters and the number of processors
  - Communication to computation ratio
  - Load balance
  - Temporal and spatial locality
- Fixed size problem -> problematic
- Understand : **How problem size influences performance evaluation?**
- Desirable: **Scale problem size as machine sizes grow**

# Terms

- **Scale a machine** = make it more (or less) powerful
  - Scale up: add cores
  - Scale down: remove cores
  - Does the architecture's  performance scale?

- **Problem size** = specific problem instance or input configuration
  - Vector of input parameters: Ocean $V = (N, e, dt, T)$
  - Different from *data size* or *memory usage*

# Key issues in scaling

- ***Under what constraints should the problem be scaled?***
  - some property must be kept fixed as the machine scales
  - Ex: data set, execution times, matrix rows per processor
- ***How should the problem be scaled?***
  - *Problem size determined by more than one parameter*
  - *Change the problem parameters to meet the constraint*
- Simplification on problem size = N (one parameter)

# Scaling models

- Properties used as basis for scaling constraints:
  - User-oriented: matrix rows per processor, transactions per processor,…
  - Resource-oriented: execution time, used memory,…

    ->more general, can be used across domains

- Resource-oriented models:
  - Problem constrained (PC): fixed problem size
  - Time constrained (TC): fixed execution time of program
  - Memory constrained (MC): fixed memory usage per processor

# Problem constrained scaling

- Fixed problem size
- Focus: **use machine to solve the same problem faster**

$$speedup = \frac{time(1proc)}{time(Pproc)}$$

- Problems:
  - Small problems on large machines
  - Large problems on too small machines
- Examples?

# Time constrained scaling

- Fixed execution time
- Focus: **complete more work in a fixed amount of time**

$$speedup = \frac{WorkDoneByPprocs}{WorkDoneBy1proc}$$

- How to measure work?
  - Execution time on a single processor (problem: superlinear speedup)
  - **Ideally, a measure of work is:**
    - **Simple to understand (architecture independent)**
    - **Scales linearly with sequential run time**

# Time constrained scaling examples

- Real-time 3D graphics: more compute power allows for rendering of much more complex scene
- Modern web sites
  - want to generate complex page, respond to user in X milliseconds
  - studies show site usage directly corresponds to page load latency

# Memory constrained scaling

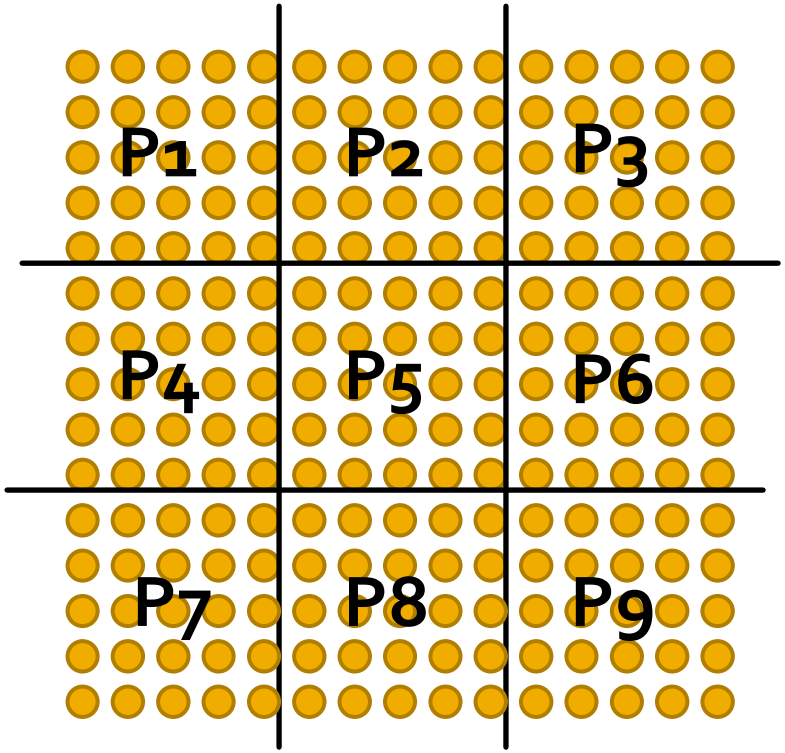
- Fixed memory usage per processor
- Focus: **run the largest problem possible without overflowing main memory**

$$speedup = \frac{WorkPerUnitTimeOnPprocs}{WorkPerUnitTimeOn1proc}$$

- Assumptions: memory resources scale with processor count
- Observations:
  - Superlinear speedup
  - Largest  speedup
  - Execution time (parallel execution) can become too large
- Example: very large scientific computations, physical simulation

# Solver example (PC)

- For N x N grid:
  - Memory requirement: $O(N^2)$
  - Total work: $O(N^2)$ grid elements x O(N) convergence $=O(N^3)$
- **N is fixed**
- Execution time: O(1/P)
- Elements per processor: $N^2/P$
- Available concurrency: fixed P
- **Comm-to-comp ratio: $O(P^{1/2})$**



# Solver example (TC)

- **Execution time: fixed at $O(N^3)$**
- Let scaled grid size be K x K
- Assume linear speedup: $K^3/P = N^3$ (so $K = NP^{1/3}$)
- Elements per processor: $K^2/P = N^2/P^{1/3}$
- Communication per processor: $K/P^{1/2} = O(1/P^{1/6})$
- **Comm-to-comp ratio: $O(P^{1/6})$**

# Solver example (MC)

- **Elements per processor: fixed ($N^2$)**
- Let scaled grid size be $NP^{1/2}$ x $NP^{1/2}$
- Execution time: $O((NP^{1/2})^3/P) = O(P^{1/2})$
- **Comm-to-comp ratio: fixed at 1/N**

- **Notice:**
  - **Best speedup**
  - **Execution time increases**

# Summary

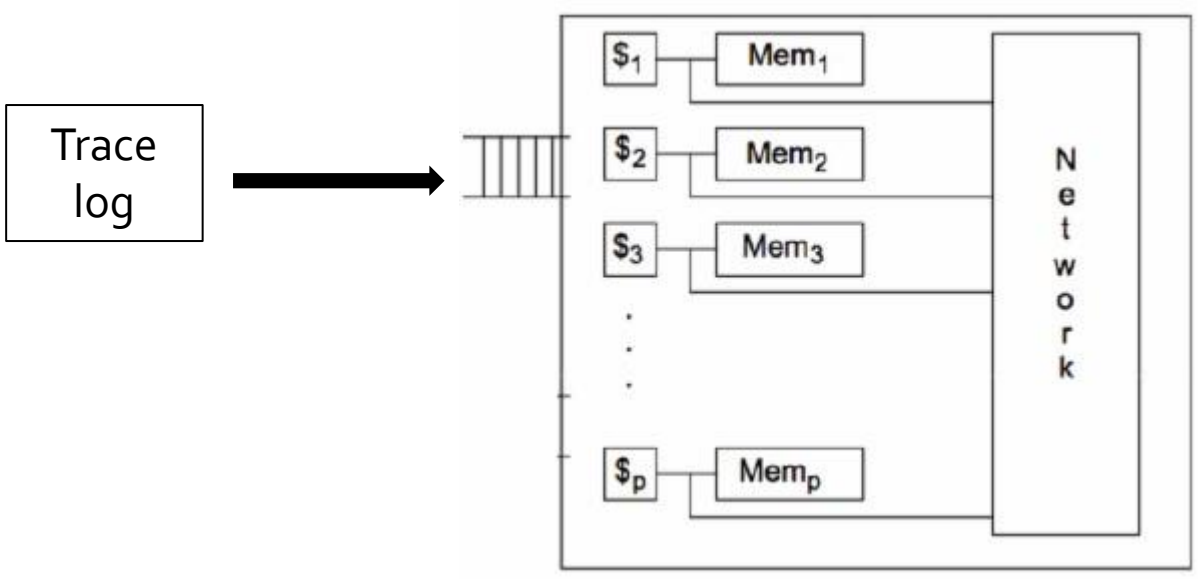
- Performance improvement due to parallelism is measured by speedup
- Speedup metrics take different forms for different scaling models
- Which model matters most is application/context specific
- Behavior of a parallel program depends significantly on the scaling properties of the problem and also the machine

# Evaluating an architectural idea: simulation

- Architects evaluate architectural decisions quantitatively using hardware performance simulators
  - Evaluate new feature:
    - Run simulation with new feature
    - Run simulation without new feature
    - Compare
  - Simulate against a wide collection of benchmarks
- Design detailed simulator to test new architectural feature
  - Detailed simulation  is expensive
  - Often cannot simulate full machine configurations or realistic problem sizes (must scale down workloads significantly!)
  - Does scaled-down simulation predict reality?

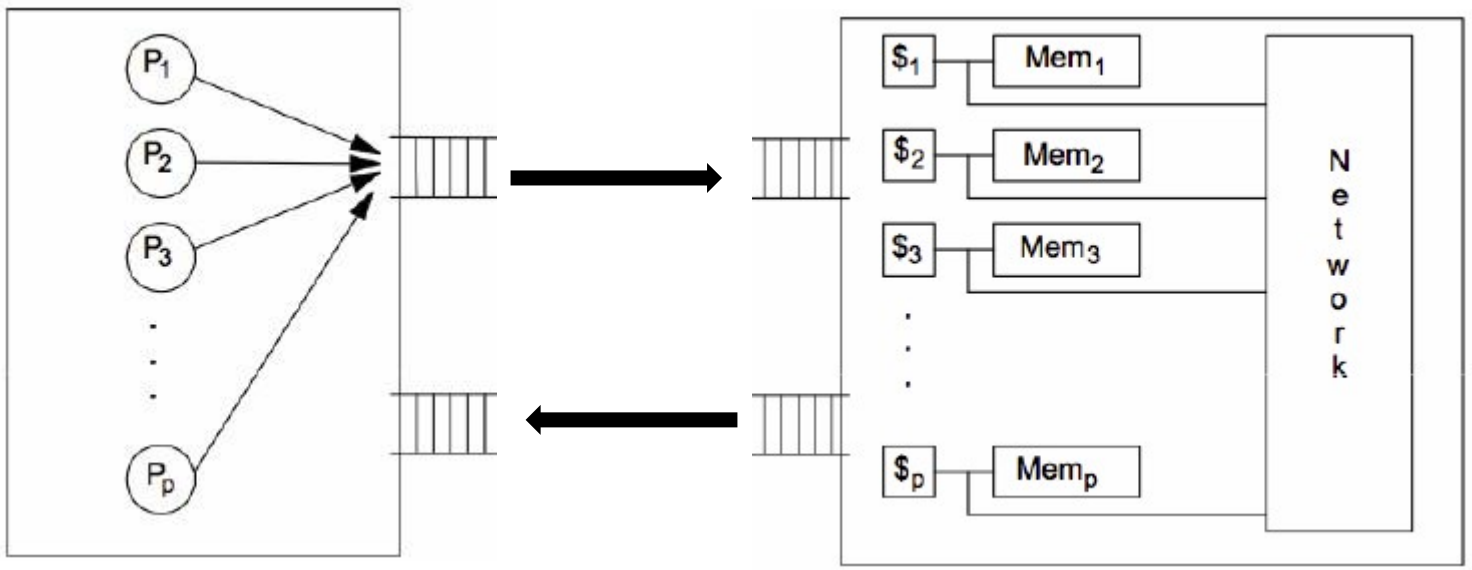
# Trace-drive simulation

- Record memory access during real execution
  - Modify the program to keep trace of memory access
  - Use tools (Intel's PIN)
- Play the recording on simulator



# Execution-driven simulation

- Executes simulated program in software
  - Simulated processors generate memory references, which are processed by the simulated memory hierarchy
- Performance of simulator is typically inversely proportional to level of simulated detail
  - How detailed should the simulation be?



# Conclusions

- Tricky to evaluate and tune parallel software and parallel machines
- Easy to obtain misleading results
- Start by precisely stating your application performance goals
- Determine if your evaluation approach is consistent with these goals

# Guidelines for evaluating performance of parallel SW

- Try the simplest parallel solution first, then measure performance
- Determine if your performance is limited by computation, memory bandwidth (or memory latency), or synchronization
- What's the best you can do in practice?

# Roofline model

- Use microbenchmarks to compute peak performance (computation, memory bandwidth)
- Compare application's performance to known peak values
- Measure:
  - Only computation
  - Only data access
  - Increase of performance based on data locality
  - Program without synchronization

# Use profilers/performance monitoring tools

- All modern processors have low-level event "performance counters"
  - details such as: instructions completed, clock ticks, L2/L3 cache hits/misses, bytes read from memory controller, etc.
- Profilers: Intel VTune

# Questions

- What does TC scaling model assume?
- What extra work is required in the case of parallel execution of an algorithm compared to the execution of the sequential algorithm?